Centralized API Management Platform

Secure Access Control, Traffic Regulation and Observability for Microservice APIs

Group Members:
Animesh Shukla, Govind Bhatter, Santanu Maharana, Vedansh Sawner

Software Project Management CS-447

August 2026

Outline

1. Introduction
2. Problem Statement
3. Aim and Objectives
4. System Architecture
5. Request Flow
6. Methodology
7. References

Introduction

- Modern applications are no longer built as a single program. They are split into many small, independent **microservices** that talk to each other over APIs.
- A mid-sized product can easily expose **dozens of APIs**, each written by a different team, deployed separately, and updated on its own schedule.
- Every one of those services then has to solve the same problems again: *Who is calling me? Are they allowed to? Are they calling too often? What happened during this request?*
- This duplicated effort is inconsistent, hard to audit, and expensive to maintain.
- An **API Management Platform** solves this by placing a single controlled entry point in front of all services.

Industry Problem

Problem Statement

Software development teams lack a centralized, secure way to manage, monitor and control access to their microservice APIs, leading to inconsistent security, unregulated traffic, and no reliable record of system activity.

Consequences observed in practice:

- **Scattered security** — authentication logic is rewritten in every service, so a single weak implementation exposes the system.
- **Unregulated traffic** — one misbehaving or malicious client can overwhelm a service and degrade it for everyone.
- **No visibility** — teams cannot answer who called an API, how often, or which calls failed.
- **Untracked changes** — configuration and access rights change without an audit trail, making incidents hard to investigate.

Aim

Aim

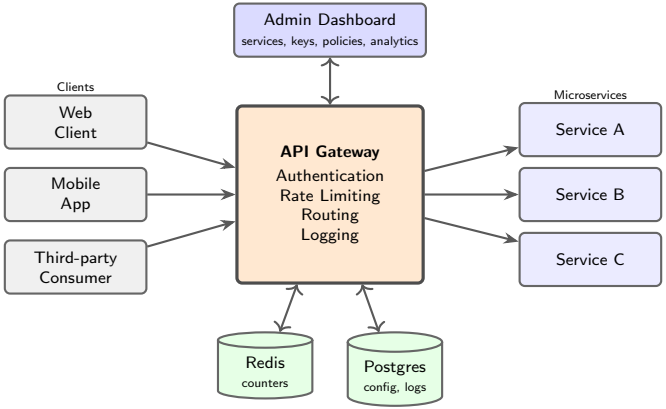
To design and develop a centralized API Management Platform that acts as a single secure gateway for an organization's microservice APIs — providing controlled access, traffic regulation, and complete visibility of system activity through a unified administrative interface.

One entry point. Consistent policy. Complete visibility.

Objectives

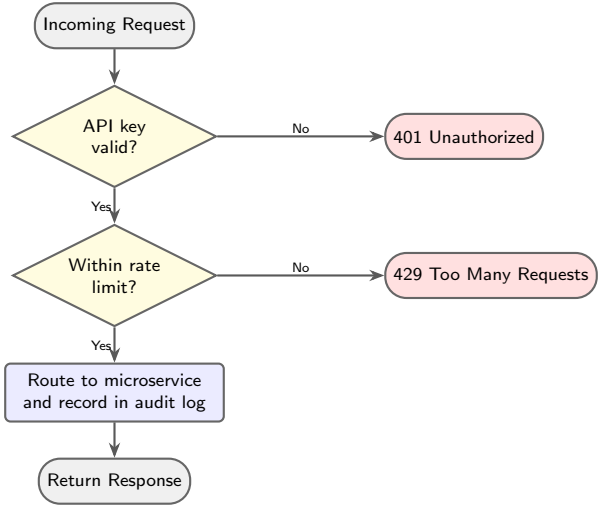
1. To build an **API gateway** that receives all client requests and forwards them to the correct backend microservice.
2. To implement **authentication** using issued API keys, so that only registered clients can access protected services.
3. To enforce configurable **rate limiting**, restricting each client to an allowed number of requests within a time window.
4. To maintain a **request log and audit trail** recording every call and every configuration change made on the platform.
5. To provide an **administrative dashboard** for registering services, issuing keys, setting policies and viewing traffic analytics.
6. To **validate** the platform against functional and load-based test cases demonstrating correct policy enforcement.

Proposed System Architecture



All external traffic passes through the gateway; no microservice is exposed directly.

Flowchart: Request Lifecycle



Every request is authenticated and throttled before it is allowed to reach a backend service.

Methodology — How We Solve It

We follow an **incremental development model**: each iteration adds one working capability to the gateway.

1. **Requirement Analysis** — identify actors (API developer, platform administrator, client application) and document functional and non-functional requirements in the SRS.
2. **Design** — define the architecture, database schema for services, keys and policies, and the request-processing pipeline.
3. **Iteration 1 — Routing** — build the reverse proxy that registers services and forwards requests to them.
4. **Iteration 2 — Security** — add API key issuance, validation and revocation.
5. **Iteration 3 — Traffic Control** — implement rate limiting using shared counters in Redis with a sliding time window.
6. **Iteration 4 — Observability** — add request logging, the configuration audit trail, and the analytics dashboard.
7. **Testing and Validation** — unit and integration tests, plus load testing to prove that limits are enforced exactly.

Technology Stack

Component	Technology
API Gateway (proxy)	Node.js with Fastify
Admin Dashboard	Next.js, Tailwind CSS
Configuration & Logs	PostgreSQL
Rate-limit Counters	Redis
Sample Microservices	Express.js
Deployment	Docker, Docker Compose
Load Testing	k6
Version Control & CI	Git, GitHub, GitHub Actions
Project Tracking	Jira (sprints, burndown)

All components are open source and freely available.

References

[1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[2] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2019.

[3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.

[4] Kong Inc., "Kong Gateway Documentation." [Online]. Available: `https://docs.konghq.com`

[5] NGINX, "API Gateway Deployment Patterns." [Online]. Available: `https://www.nginx.com`

[6] Redis, "Rate Limiting Patterns with Redis." [Online]. Available: `https://redis.io/docs`

Thank You

Questions?